

Оценки коммуникационной
трудоемкости
параллельных алгоритмов.

- Кроме участков кода, которые необходимо выполнять последовательно, у параллельной программы есть еще три источника накладных расходов:
 - 1) создание и сопровождение процессов;
 - 2) взаимодействие процессов;
 - 3) синхронизация процессов. Все эти расходы взаимосвязаны – и часто уменьшение влияния одного из них приводит к увеличению другого.
- Рассмотрим стандартные способы уменьшения этих расходов.

- 1. Для уменьшения расходов, связанных с сопровождением процессов, можно создавать строго один процесс на каждом вычислителе. Однако это может увеличить простои вычислителя или усложнить задачу балансировки нагрузки.
- 2. Взаимодействие процессов с помощью обмена сообщениями – это накладные расходы на инициализацию операции обмена (как у отправителя, так и у получателя) и расходы, связанные с транспортировкой сообщения по Сети. Поскольку с точки зрения отправки одного сообщения эти расходы неустранимы, то единственный способ их уменьшения – минимизация количества обменов и выполнение операции обмена, если это возможно, в фоновом режиме.

- 3. Синхронизация при программировании в ВС с распределенной памятью происходит явно при выполнении барьеров и неявно при передаче сообщений. Использование неблокирующих режимов обмена, посылка сообщений пакетами, дополнительные локальные вычисления каждым процессом вместо глобальных операций (если это возможно) – это способы уменьшения накладных расходов на синхронизацию.

Понятия среды выполнения операции обмена.

- **Латентность** – время между инициированием передачи данных при посылке и прибытия первого байта при приеме.

Латентность часто зависит от длины посылаемых сообщений. Ее значение может изменяться в зависимости от того, послано ли большое количество маленьких сообщений или нескольких больших сообщений.

Латентность характеризуется тремя параметрами:

$t_{prepare}$ – время подготовки данных;

t_{serv} – время передачи служебных данных;

t_{byte} – время, необходимое для передачи одного байта.

- **Пропускная способность** – это величина, обратная времени, необходимому для передачи одного байта ($1/t_{byte}$).

Пропускная способность обычно выражается в Мб/с (Гб/с). Пропускная способность важна, когда передаются сообщения больших размеров.

Базовые оценки трудоемкости обмена данными для кластерных систем.

Топология сети кластера редко представлена полным графом, обычно имеются определенные ограничения на одновременность выполнения коммуникационных операций. Использование метода передачи пакетов (реализуемого, как правило, на основе протокола TCP/IP) является характерным для выполнения коммуникационных операций.

- Базовой оценкой времени передачи сообщения размером m байт является модель Хокни:

$$t_{db} \sim t_{prepare} + mt_{byte}$$

- С учетом времени на передачу служебной информации:

$$t_{db} \sim t_{prepare} + mt_{byte} + t_{serv}.$$

- Однако если сообщение достаточно велико и составляет несколько пакетов, то эта оценка требует уточнения, поскольку не учитывает зависимость $t_{prepare}$ и t_{serv} от количества пакетов.

- Пусть $V_{\max}$ определяет максимальный размер пакета, который может быть доставлен в Сеть (например, для ОС MS Windows в Сети Fast Ethernet $V_{\max} = 1500$ байт); V_{serv} определяет объем служебных данных в каждом из пересылаемых пакетов (для протокола TCP/IP, ОС MS Windows и Сети Fast Ethernet $V_{serv} = 8$ байт). Тогда количество пакетов, на которое разбивается сообщение, можно характеризовать следующей величиной:

$$n \sim \left\lceil m / (V_{\max} - V_{serv}) \right\rceil.$$

Пусть также $t_{prepare}^0$

характеризует аппаратную составляющую латентности, которая зависит от параметров используемого сетевого оборудования,

а значение $t_{prepare}^1$

задает время подготовки одного байта данных для передачи по Сети.

Предположим, что латентность увеличивается линейно в зависимости от объема передаваемых данных:

$$t_{prepare} = t_{prepare}^0 + kt_{prepare}^1$$

причем подготовка данных для передачи второго и всех последующих пакетов может быть совмещена с пересылкой по Сети предшествующих пакетов, следовательно, латентность ограничена сверху:

$$t_{prepare} \sim t_{prepare}^0 + (V_{\max} - V_{serv})t_{prepare}^1.$$

- Учет также, что с увеличением количества пакетов растет время на пересылку служебной информации t_{serv} . В результате вместо оценки Хокни можно записать более точную оценку времени, требующегося для передачи сообщения размером m байт

$$t_{db} \sim \begin{cases} t_{prepare}^0 + mt_{prepare}^1 + (m + V_{serv})t_{byte}, & n = 1; \\ t_{prepare}^0 + (V_{\max} - V_{serv})t_{prepare}^1 + (m + nV_{serv})t_{byte}, & n > 1. \end{cases}$$

- В литературе представлены данные вычислительных экспериментов с использованием библиотеки MPI по изучению времени, затрачиваемого на передачу сообщений большого диапазона длин, в Сети многопроцессорного кластера Нижегородского госуниверситета им. Н. И. Лобачевского.
- Приведенные данные свидетельствуют об очень хорошей точности предсказаний с помощью вышеприведенной оценки для сообщений любой длины.
- Ранее приведенные оценки дают правдоподобные результаты при отправке большого количества сообщений большой длины. С учетом того, что они требуют минимум информации о характеристиках среды, их можно использовать для грубых оценок времени коммуникаций в параллельной программе.